```
-- create explicit EXCLUSIVE LOCK of trips table
LOCK trips IN ACCESS EXCLUSIVE MODE;

-- "psql2"
-- Inspect pg_locks view
-- Find the "AccessExclusiveLock" for "trips"
SELECT
  mode,
  pg_class.relname,
  locktype,
  relation
FROM pg_locks
JOIN pg_class
ON pg_locks.relation = pg_class.oid
AND pg_locks.mode = 'AccessExclusiveLock';

-- -[ RECORD 1 ]-----------------
-- mode     | AccessExclusiveLock
-- relname  | trips
-- locktype | relation
-- relation | 461492

-- "psql1", inspect the current lock_timeout;
-- A lock_timeout of 0 means it is disabled, so
-- a statement waiting on a lock will run forever
SHOW lock_timeout;

-- "psql2"
-- Create the DDL statement from earlier.
-- DDL statement acquires an exclusive lock
-- But it will get blocked!
ALTER TABLE trips ADD COLUMN city_id INTEGER DEFAULT 1;

-- Since this hanging go back to "psql1"

-- Query pg_stat_activity from "psql1"
-- Look for waiting transactions
-- Confirm the ALTER TABLE is listed and blocked
SELECT
  wait_event_type,
  wait_event, query
FROM pg_stat_activity
WHERE wait_event = 'relation'
AND query LIKE '%ALTER TABLE%';

-- -[ RECORD 1 ]---+-----------------------------------------------------
-- wait_event_type | Lock
-- wait_event      | relation
-- query           | ALTER TABLE trips ADD COLUMN city_id INTEGER DEFAULT 1;

-- NOTE: UNBLOCK
-- To unblock, COMMIT or ROLLBACK the original transaction from "psql1"

-- "psql1"
```